

Reporte

July 28, 2026

1 Laboratorio 1

1.1 Integrantes

Nombre	Carnet	Usuario Git
Edwin Jose Gabriel De Leon Garcia	22809	EJGDLG
Gustavo Adolfo Cruz Bardales	22779	G2309
Josué Emanuel Say Garcia	22801	JosueSay
Mathew Alexander Cordero Aquino	22982	donmatthiuz

1.2 Repositorio

[Link al Repositorio](#)

1.3 Task 1

Dado el contexto de la empresa de logística, diseñen formalmente el MDP que modela el problema. Su diseño debe especificar con precisión y justificación:

1.3.1 Inciso 1

El espacio de estados **S**: ¿qué información debe contener el estado para que la propiedad de Markov se satisfaga razonablemente? Justifiquen cada variable que incluyen y cada variable que deciden omitir. Si omiten algo que podría ser relevante, expliquen qué consecuencia tiene esa omisión sobre la validez del modelo.

Respuesta:

La pregunta que gobierna el diseño del estado no es qué información existe, sino qué información es **suficiente** para que la propiedad de Markov se aproxime razonablemente. La propiedad no es una característica del mundo, sino de la representación: si falla, el estado está mal definido, no el entorno.

Propuesta:

$$s = (x, y, b, \mathbf{d}, c, w)$$

Variables incluidas:

Variable	Dominio	Justificación
(x, y)	$\{0, \dots, 4\}^2$	Posición del dron en la cuadrícula. Sin ella no hay noción de transición espacial.
b	$\{0, 1, \dots, B\}$	Nivel de batería discretizado. Determina qué acciones son viables y la proximidad al fallo.
$\mathbf{d}$	$\{0, 1\}^k$	Vector binario de entregas pendientes sobre los k puntos designados. Codifica el progreso de la misión.
c	$\{0, 1\}$	Indicador de carga a bordo (paquete cargado o no). Distingue “voy a entregar” de “voy de regreso”.
w	$\{0, 1, 2\}$	Condición de viento o clima en tres niveles (nulo, moderado, adverso). Afecta la estocasticidad del movimiento y el consumo.

La cardinalidad resultante es $|\mathcal{S}| = 25 \cdot (B+1) \cdot 2^k \cdot 2 \cdot 3$. Con $B = 10$ y $k = 3$ se obtienen $25 \cdot 11 \cdot 8 \cdot 2 \cdot 3 = 13,200$ estados: un espacio tabular perfectamente manejable para evaluación exacta de políticas.

El punto crítico es $\mathbf{d}$. Sin el vector de entregas pendientes, dos visitas a la misma coordenada en momentos distintos de la misión serían indistinguibles pese a tener valores completamente diferentes.

Variables omitidas y sus consecuencias:

- **Historial de trayectoria.** La posición actual más el vector de entregas ya resumen el pasado relevante para decidir el siguiente movimiento. Conservarla haría explotar $|\mathcal{S}|$ sin ganancia informativa.
- **Tiempo transcurrido absoluto.** El costo temporal se internaliza vía recompensa negativa por paso, sin necesidad de estado adicional. **Consecuencia:** el modelo no puede representar ventanas de entrega con hora límite; si el negocio las exige, habría que añadir un componente temporal y el espacio crecería linealmente en el número de intervalos.
- **Posición de otros drones.** Si los drones comparten espacio aéreo, el entorno se vuelve no estacionario desde la perspectiva de cada agente individual y la propiedad de Markov se rompe. Es la limitación más severa del diseño y se aborda en la sección de conclusiones.
- **Tráfico aéreo o zonas restringidas dinámicas.** Si las restricciones cambian durante el episodio, el agente las percibirá como ruido inexplicable.
- **Peso o tipo específico del paquete.** Si el consumo de batería dependiera fuertemente del peso, habría que incorporarlo a s o el consumo real diferiría sistemáticamente del modelado.

Regla general: se incluye una variable si su omisión hace que dos situaciones con valores esperados distintos colapsen en el mismo estado.

1.3.2 Inciso 2

El espacio de acciones **A**: definan las acciones disponibles para el drone. ¿Es discreto o continuo? ¿Hay acciones que deberían restringirse en ciertos estados? ¿Cómo modelan esa restricción dentro del MDP?

Respuesta:

El espacio de acciones es discreto y finito, con siete elementos:

$$\mathcal{A} = \{\text{Norte, Sur, Este, Oeste, Entregar, Recargar, Esperar}\}$$

- Las cuatro acciones de movimiento desplazan el drone una casilla.
- **Entregar** libera el paquete si el drone está sobre un punto de entrega pendiente con $c = 1$.
- **Recargar** restaura la batería, disponible únicamente en la base.
- **Esperar** mantiene la posición; es racional bajo viento adverso, cuando moverse tiene alta probabilidad de desvío.

Se elige discreto porque la cuadrícula ya impone una discretización natural del espacio, y un espacio de acciones finito permite evaluación tabular exacta, que es precisamente lo que la empresa quiere validar antes de invertir en aproximación funcional.

No todas las acciones son válidas en todo estado. Se define $\mathcal{A}(s) \subseteq \mathcal{A}$ como el conjunto de acciones admisibles. Existen dos formas de modelarlo dentro del MDP:

1. **Restricción estructural.** Definir $\pi(a | s) = 0$ para toda $a \notin \mathcal{A}(s)$ y normalizar sobre las acciones válidas; el agente nunca considera lo imposible.
2. **Restricción por transición y penalización.** Mantener $\mathcal{A}$ completo y hacer que las acciones inválidas transicionen al mismo estado con recompensa fuertemente negativa. El agente aprende la restricción en lugar de recibirla.

1.3.3 Inciso 3

La función de recompensa $\mathbf{r}(s, \mathbf{a}, s')$: diseñen una función de recompensa que capture el objetivo real de la empresa. Consideren al menos tres objetivos potencialmente conflictivos: eficiencia de entrega, consumo de batería y seguridad de vuelo. ¿Cómo ponderan esos objetivos? ¿Qué consecuencias tendría una ponderación incorrecta sobre el comportamiento del agente?

Respuesta:

El agente optimiza lo que se le pidió, no lo que se pensó que se pidió; si la recompensa no captura el objetivo real, el agente encontrará formas de maximizarla que no anticipamos, y eso es responsabilidad de quien diseña, no del algoritmo.

Se plantean tres objetivos en tensión: **eficiencia de entrega, consumo de batería y seguridad de vuelo**. La función propuesta es aditiva ponderada:

$$r(s, a, s') = w_e \cdot r_{\text{entrega}} + w_t \cdot r_{\text{tiempo}} + w_b \cdot r_{\text{bateria}} + w_s \cdot r_{\text{seguridad}}$$

Componentes y valores propuestos

Componente	Evento	Valor base	Peso
r_{entrega}	Entrega completada con éxito	+50	$w_e = 1.0$
r_{entrega}	Misión completa (todas las entregas + retorno a base)	+100	$w_e = 1.0$
r_{tiempo}	Cada paso no terminal	-1	$w_t = 1.0$
$r_{\text{batería}}$	Consumo por movimiento	-0.5 por unidad	$w_b = 1.0$
$r_{\text{batería}}$	Batería agotada en vuelo (pérdida del dron)	-200	$w_b = 1.0$
$r_{\text{seguridad}}$	Colisión con obstáculo o edificio	-500	$w_s = 1.0$
$r_{\text{seguridad}}$	Vuelo con $w = 2$ (viento adverso)	-5	$w_s = 1.0$

Los valores están escalados de modo que el costo de un fallo catastrófico domine por un orden de magnitud sobre cualquier ganancia acumulable por eficiencia. Una entrega vale +50; una colisión vale -500. Un agente racional nunca cambiará diez entregas por un accidente.

Consecuencias de una ponderación incorrecta

La ponderación no es un detalle de ajuste, es la especificación del objetivo del negocio:

- **Sobrepeso de eficiencia (w_t o w_e demasiado alto).** El dron toma rutas directas sobre zonas peligrosas y vuela con batería crítica para ganar unos pasos. Optimiza la métrica de entrega y destruye la flota.
- **Sobrepeso de batería (w_b demasiado alto).** El dron prefiere quedarse en la base recargando indefinidamente antes que arriesgar consumo. Si $|r_{\text{batería}}|$ por movimiento supera el retorno descontado de una entrega, la política óptima es literalmente no despegar.
- **Sobrepeso de seguridad (w_s demasiado alto).** El agente adopta una política ultraconservadora: espera indefinidamente cuando hay viento y solo se mueve en condiciones ideales. Cero accidentes, cero entregas.
- **Bonificación mal especificada.** Si se premia “acercarse al punto de entrega” en lugar de “entregar”, el agente puede aprender a oscilar cerca del objetivo acumulando recompensa sin completar nunca la tarea. Es el caso clásico de *reward hacking*.

1.3.4 Inciso 4

La función de transición $\mathbf{p}(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})$: ¿es determinista o estocástica? Si es estocástica, ¿qué fuentes de aleatoriedad existen en el dominio real y cómo las modelan? Escriban al menos tres transiciones concretas con sus probabilidades y justifiquen cada valor.

Respuesta:

Produciría un agente que planifica rutas óptimas bajo un supuesto de control perfecto y falla sistemáticamente en operación real.

Fuentes de aleatoriedad en el dominio

1. **Viento y turbulencia urbana.** Los cañones entre edificios generan ráfagas que desvían el dron de la trayectoria comandada.
2. **Error de actuación y sensado.** Los motores y el GPS tienen tolerancia; el desplazamiento real no coincide exactamente con el comandado.
3. **Fallo de entrega.** El punto de entrega puede estar obstruido o el receptor ausente, dejando el paquete a bordo.
4. **Consumo variable de batería.** Depende de la carga instantánea del motor, que varía con las condiciones.
5. **Cambio de condición climática.** w evoluciona durante el episodio según su propia dinámica.

Transiciones concretas

Transición 1 — Movimiento bajo viento nulo ($w = 0$), acción Norte desde $(2, 2)$:

$$p((2, 1) \mid (2, 2), \text{Norte}) = 0.95, \quad p((1, 2) \mid \cdot) = 0.025, \quad p((3, 2) \mid \cdot) = 0.025$$

El 5% residual reparte simétricamente el error de actuación entre las dos casillas laterales, reflejando que el desvío no tiene dirección preferente en ausencia de viento. No se asigna probabilidad al retroceso porque un dron no se desplaza en sentido contrario al comando.

Transición 2 — Movimiento bajo viento adverso ($w = 2$), acción Norte desde $(2, 2)$:

$$p((2, 1) \mid \cdot) = 0.70, \quad p((1, 2) \mid \cdot) = 0.15, \quad p((3, 2) \mid \cdot) = 0.10, \quad p((2, 2) \mid \cdot) = 0.05$$

Bajo viento adverso la fiabilidad cae a 0.70. La asimetría entre las laterales (0.15 frente a 0.10) modela una dirección dominante del viento, información que es observable y por tanto legítima de incorporar. El 5% de permanencia representa el caso en que la ráfaga cancela el avance. Estos valores son calibrables con telemetría real: la estructura del modelo es la contribución, los números concretos son una hipótesis inicial.

Transición 3 — Acción Entregar sobre un punto designado con $c = 1$:

$$p(\mathbf{d}' = \mathbf{d} \setminus \{i\}, c' = 0 \mid s, \text{Entregar}) = 0.90, \quad p(\mathbf{d}' = \mathbf{d}, c' = 1 \mid s, \text{Entregar}) = 0.10$$

Un 10% de fallo de entrega recoge las causas operativas habituales: zona de aterrizaje obstruida, receptor ausente, rechazo del paquete. La consecuencia de modelarlo es que el agente aprende a considerar reintentos en su planificación en lugar de asumir que una visita equivale a una entrega.

Transición 4 — Consumo de batería (aplicable a toda acción de movimiento):

$$p(b' = b - 1 \mid \cdot) = 0.80, \quad p(b' = b - 2 \mid \cdot) = 0.20$$

El consumo no es constante. Ese 20% de consumo doble es lo que induce al agente a mantener un margen de seguridad en lugar de planificar al límite exacto de autonomía. Un modelo de consumo determinista produciría un agente que se queda sin batería con regularidad.

Todas las distribuciones cumplen $\sum_{s'} p(s' \mid s, a) = 1$ y, por construcción, dependen solo de (s, a) : la propiedad de Markov se preserva.

1.3.5 Inciso 5

El factor de descuento : propongan un valor y justifiquenlo en términos del problema, no solo en términos matemáticos.

Respuesta:

Se propone $\gamma = 0.95$.

El horizonte efectivo de un factor de descuento es aproximadamente $1/(1 - \gamma)$, lo que con $\gamma = 0.95$ da unos **20 pasos**. En una cuadrícula de 5×5 , una misión típica de tres entregas con retorno a base ocupa entre 15 y 25 pasos. El agente valora hoy la recompensa de completar el recorrido y regresar, en vez de descartarla por lejana.

Los extremos ilustran por qué el valor importa:

- γ **bajo (por ejemplo 0.5, horizonte ≈ 2 pasos)**. El agente se vuelve miope. Toma la entrega más cercana, ignora que quedarse sin batería a mitad de ruta cuesta -200 y nunca aprende que regresar a la base tiene valor. Optimiza el siguiente movimiento, no la misión.
- $\gamma = 1$ **(sin descuento)**. Es defendible porque el episodio es finito y termina en un estado terminal, igual que en el GridWorld canónico. Sin embargo, si existe cualquier probabilidad de que el episodio no termine (un drone que orbita indefinidamente sin agotar batería), la suma de retornos diverge y la evaluación de políticas no converge. $\gamma = 0.95$ garantiza convergencia sin sacrificar la visión de misión completa.
- γ **muy alto (0.999, horizonte ≈ 1000)**. Innecesario. Introduce lentitud de convergencia sin aportar información relevante en un problema cuyo episodio dura decenas de pasos.

Hay además una lectura de negocio: γ codifica cuánto vale para la empresa una entrega futura frente a una inmediata. Un γ de 0.95 dice que sí importa terminar la ruta completa, pero que la demora tiene un costo real.

Diagrama MDP

flowchart LR

```
S["Estado s = (x, y, b, d, c, w)"] --> PI["Politica pi(a|s)"]
PI --> A["Accion en A(s)"]
A --> P["Transicion estocastica p"]
P --> S
P --> SP["Estado siguiente s prima"]
P --> R["Recompensa ponderada r"]
R --> V["Retorno descontado con gamma = 0.95"]
SP --> S
```

1.4 Task 2

Respondan las siguientes preguntas con argumentación técnica

1.4.1 Inciso 1

Identifiquen al menos dos situaciones en las que la propiedad de Markov se violaría con su representación de estado actual. Para cada una, propongan una extensión del estado que restaure la propiedad y discutan el costo computacional de esa extensión.

Respuesta:

Primera Situación: Varios drones comparten el mismo espacio aereo. Si un dron que lo llamaremos 1 llega al mismo estado $s = (2, 2, b, \mathbf{d}, c, w)$. En el primer caso, un dron 2 está a una casilla de distancia y convergiendo hacia la misma celda; en el segundo, 2 está en la esquina opuesta del grid.

El estado de 1 es idéntico a ambos casos pero $P(s' | s, a)$ es radicalmente distinto: En el primer caso hay riesgo que se choquen y la acción óptima es desviarse. En el segundo ese riesgo no existe, por lo tanto se puede decir lo siguiente = “Dos estados idénticos con futuros condicionales distintos = violación directa de Markov”

Segunda Situación: Ventanas de entrega con hora límite Supongamos que el punto de entrega i tiene un plazo: debe completarse antes del paso 15. Dos episodios llegan al mismo $s = (x, y, b, \mathbf{d}, c, w)$ con i aún pendiente, uno en el paso 5 y otro en el paso 18. El estado observado es idéntico, pero en el primer caso todavía es viable cumplir el plazo y en el segundo ya no.

1.4.2 Inciso 2

En el dominio de logística urbana, ¿qué tan razonable es el supuesto de que el agente puede observar el estado completo? ¿Qué variables del estado real probablemente no son directamente observables por el dron? ¿Cómo afecta eso la validez del modelo MDP versus un modelo POMDP?

Respuesta:

El supuesto de observabilidad completa es la pieza más frágil del modelo MDP tal como se planteó en la Tarea 1. Un MDP asume que el agente conoce $s = (x, y, b, \mathbf{d}, c, w)$ sin ruido y sin retraso en cada paso; en un dron real eso es, en el mejor de los casos, una aproximación, y en el peor, una ficción cómoda para poder usar programación dinámica tabular.

En pocas palabras de las seis variables del estado, solo $\mathbf{d}$ y (con reservas) c son observables sin ruido relevante. (x, y) y b son observables con error acotado y tratable. w es la que más se aleja de “observable”: el dron infiere una condición global a partir de una medición local.

Debido a estas limitaciones, el modelo MDP pierde validez como representación fiel del problema, ya que no se cumple el supuesto de estado completamente observable. Debido a esto, el problema se ajusta mejor a un modelo POMDP, donde el agente debe tomar decisiones basándose en observaciones parciales y mantener una creencia sobre el estado real del sistema. El uso de un MDP en este contexto es, por tanto, colocar de manera sencilla desde el punto de vista computacional, pero que introduce una diferencia notable entre el modelo y la realidad.

1.4.3 Inciso 3

Argumenten si este problema debería modelarse como una tarea episódica o continua. ¿Cómo cambia esa decisión el diseño de la función de recompensa y el valor de γ ?

Respuesta:

Es episódica esto porque: El propio diseño de la Tarea 1 ya asume esto implícitamente: - La recompensa (Inciso 3) tiene eventos que solo tienen sentido como cierre de episodio: “misión completa” (+100) y “batería agotada en vuelo / pérdida del dron” (−200). Ambos son bonificaciones/penalizaciones que ocurren una vez y presuponen un estado terminal absorbente. - La

justificación de γ (Inciso 5) ya lo dice explícitamente: “ $\gamma = 1$ es defendible porque el episodio es finito y termina en un estado terminal, igual que el GridWorld canónico.”

Si el problema no fuera episódico sino continuo, sería necesario utilizar un valor de γ menor que 1 para garantizar la convergencia del retorno esperado y evitar que la suma de recompensas se vuelva infinita. En este tipo de entornos, no se puede depender de recompensas terminales únicas, ya que no existe un estado final definido; en su lugar, la función de recompensa debe diseñarse con señales más locales y recurrentes que guíen el comportamiento del agente a lo largo del tiempo.

1.5 Task 3

Implementen en Python un evaluador iterativo de políticas para un GridWorld 5x5 que represente la ciudad de drones. Su implementación debe:

1.5.1 Inciso 1

Representar el MDP completo: estados, acciones, función de transición y función de recompensa, consistentes con el diseño de la Tarea 1. No usen librerías de RL como Gymnasium. El MDP debe estar implementado desde cero.

Nota de simplificación. La Tarea 1 definió $s = (x, y, b, \mathbf{d}, c, w)$, con $|\mathcal{S}| \approx 13,200$. El Inciso 4 de esta tarea exige *una sola cuadrícula 5x5* coloreada por $V^\pi(s)$ por política, lo cual solo es representable de forma directa si el estado colapsa a la posición (x, y) , es decir, 25 estados. Aquí reducimos el estado a la celda del dron y trasladamos batería, viento y entregas a **propiedades fijas del terreno** en vez de dimensiones del estado:

Elemento	Celdas	Rol
Edificios (obstáculo)	$(0, 3), (1, 1), (1, 3), (3, 2)$	Intentar entrar produce colisión: rebote + $r_{\text{seguridad}} = -500$, igual que Tarea 1 Inciso 3.
Puntos de entrega (terminal)	$(0, 4), (4, 4)$	Absorbentes; entrar otorga $r_{\text{entrega}} = +50$.
Base	$(4, 0)$	Origen nominal del dron (referencia visual).
Celdas con viento adverso	$(2, 2), (3, 3)$	Transición más ruidosa y penalización de vuelo $r_{\text{seguridad}} = -5$, igual que Tarea 1 Inciso 3/4.

Esta es la misma limitación que se discute en la Tarea 2 Inciso 1: omitir una variable rompe Markov si su efecto es heterogéneo entre estados que colapsan. Aquí el efecto de b , $\mathbf{d}$ y w se vuelve **determinista y ligado a la celda** en lugar de una dimensión del estado — una simplificación deliberada para cumplir el requisito de visualización del Inciso 4, y que se retoma como limitación explícita en la Tarea 4.

El resto del diseño reutiliza los valores numéricos exactos de la Tarea 1: probabilidades de transición del Inciso 4 (0.90/0.05/0.05 en calma, 0.70/0.15/0.10/0.05 con viento adverso) y componentes de recompensa del Inciso 3 (-1 por paso, $+50$ entrega, -500 colisión, -5 vuelo con viento adverso).


```

[2]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from collections import deque

# -----
# Espacio de estados y acciones
# -----

GRID_SIZE = 5

NORTH, SOUTH, EAST, WEST, WAIT = range(5)
ACTIONS = [NORTH, SOUTH, EAST, WEST, WAIT]
ACTION_NAMES = {NORTH: "Norte", SOUTH: "Sur", EAST: "Este", WEST: "Oeste", WAIT:
    ↪ "Esperar"}
ACTION_DELTA = {NORTH: (-1, 0), SOUTH: (1, 0), EAST: (0, 1), WEST: (0, -1),
    ↪ WAIT: (0, 0)}
LATERAL = {NORTH: (EAST, WEST), SOUTH: (EAST, WEST), EAST: (NORTH, SOUTH), WEST:
    ↪ (NORTH, SOUTH)}

# Terreno: mismas semánticas y costos que la Tarea 1, fijados por celda.
OBSTACLES = {(0, 3), (1, 1), (1, 3), (3, 2)} # edificios
DELIVERIES = {(0, 4), (4, 4)} # puntos de entrega (terminales)
BASE = (4, 0) # base (referencia visual)
WIND_CELLS = {(2, 2), (3, 3)} # viento adverso (w = 2)

# Recompensa: valores idénticos a la Tarea 1, Inciso 3.
R_STEP = -1.0
R_DELIVERY = 50.0
R_COLLISION = -500.0
R_WIND_FLIGHT = -5.0

# Transición: probabilidades idénticas a la Tarea 1, Inciso 4.
P_MAIN_CALM, P_SIDE_CALM = 0.90, 0.05
P_MAIN_WIND, P_SIDE1_WIND, P_SIDE2_WIND, P_STAY_WIND = 0.70, 0.15, 0.10, 0.05

class DroneGridWorldMDP:
    """MDP tabular desde cero: sin Gymnasium ni librerías de RL."""

    def __init__(self):
        self.size = GRID_SIZE
        self.states = [(r, c) for r in range(self.size) for c in range(self.
            ↪ size)]

        self.actions = ACTIONS
        self.terminal_states = set(DELIVERIES)

```

```

def is_terminal(self, s):
    return s in self.terminal_states

def _in_bounds(self, r, c):
    return 0 <= r < self.size and 0 <= c < self.size

def _attempt(self, s, direction):
    """Resultado de intentar moverse en `direction` desde `s`, ya
    resolviendo bordes y edificios (ambos producen rebote)."""
    dr, dc = ACTION_DELTA[direction]
    r, c = s[0] + dr, s[1] + dc
    if not self._in_bounds(r, c) or (r, c) in OBSTACLES:
        return s
    return (r, c)

def _outcomes(self, s, a):
    """Sub-eventos físicos (prob, dirección_intentada) de ejecutar `a`
    en `s`. `None` = la ráfaga de viento cancela el avance."""
    if self.is_terminal(s) or a == WAIT:
        return [(1.0, WAIT)]
    d1, d2 = LATERAL[a]
    if s in WIND_CELLS:
        return [(P_MAIN_WIND, a), (P_SIDE1_WIND, d1),
                (P_SIDE2_WIND, d2), (P_STAY_WIND, None)]
    return [(P_MAIN_CALM, a), (P_SIDE_CALM, d1), (P_SIDE_CALM, d2)]

def reward(self, s, a, s_next, attempted_direction):
    """r(s,a,s'): -1 por paso, +50 al entregar, -500 si el sub-evento
    chocó contra un edificio, -5 adicional por volar con viento adverso."""
    if self.is_terminal(s):
        return 0.0
    if s_next in DELIVERIES:
        return R_DELIVERY
    r = R_STEP
    if attempted_direction not in (None, WAIT) and s_next == s:
        dr, dc = ACTION_DELTA[attempted_direction]
        target = (s[0] + dr, s[1] + dc)
        if self._in_bounds(*target) and target in OBSTACLES:
            r += R_COLLISION
    if s in WIND_CELLS and a != WAIT:
        r += R_WIND_FLIGHT
    return r

def step_distribution(self, s, a):
    """Lista de (prob, s', r(s,a,s')) - la entrada que usa el operador
    de Bellman. La recompensa se calcula por sub-evento físico, no
    sobre el estado agregado, para no perder el riesgo de colisión de

```

```

        una deriva lateral."""
    result = []
    for p, direction in self._outcomes(s, a):
        s_next = s if direction in (None, WAIT) else self._attempt(s,
↪direction)
        r = self.reward(s, a, s_next, direction)
        result.append((p, s_next, r))
    return result

    def transitions(self, s, a):
        """p(s'|s,a) agregada por estado destino (inspección del modelo)."""
        probs = {}
        for p, s_next, _ in self.step_distribution(s, a):
            probs[s_next] = probs.get(s_next, 0.0) + p
        return list(probs.items())

mdp = DroneGridWorldMDP()
print(f"|S| = {len(mdp.states)} estados, |A| = {len(mdp.actions)} acciones")
print("Transición p(s'|(2,2),Norte) en celda de viento:", mdp.transitions((2,
↪2), NORTH))
print("Transición p(s'|(2,0),Norte) en celda en calma: ", mdp.transitions((2,
↪0), NORTH))

```

|S| = 25 estados, |A| = 5 acciones

Transición p(s'|(2,2),Norte) en celda de viento: [((1, 2), 0.7), ((2, 3), 0.15), ((2, 1), 0.1), ((2, 2), 0.05)]

Transición p(s'|(2,0),Norte) en celda en calma: [((1, 0), 0.9), ((2, 1), 0.05), ((2, 0), 0.05)]

1.5.2 Inciso 2

Implementar la evaluación iterativa de políticas mediante aplicación repetida del operador de Bellman hasta convergencia. El criterio de convergencia debe ser configurable: el algoritmo termina cuando el cambio máximo en cualquier valor de estado entre dos iteraciones consecutivas es menor que un umbral .

Aplicación *synchronous* del operador de Bellman de evaluación:

$$V_{k+1}(s) = \sum_a \pi(a | s) \sum_{s'} p(s' | s, a) [r(s, a, s') + \gamma V_k(s')]$$

Se detiene cuando $\max_s |V_{k+1}(s) - V_k(s)| < \theta$, con θ y γ configurables.

```

[3]: def policy_evaluation(mdp, policy, gamma=0.95, theta=1e-6,
↪max_iterations=10000):
    """Evaluación iterativa de políticas por aplicación repetida del
    operador de Bellman. `policy[s]` es un dict {accion: prob}.

```

```

Retorna (V, numero_de_iteraciones_hasta_convergencia)."""
V = {s: 0.0 for s in mdp.states}
for it in range(1, max_iterations + 1):
    delta = 0.0
    V_new = {}
    for s in mdp.states:
        if mdp.is_terminal(s):
            V_new[s] = 0.0
            continue
        v = 0.0
        for a, pi_a in policy[s].items():
            if pi_a == 0.0:
                continue
            v += pi_a * sum(p * (r + gamma * V[s_next])
                            for p, s_next, r in mdp.step_distribution(s,
↪a))
        V_new[s] = v
        delta = max(delta, abs(V_new[s] - V[s]))
    V = V_new
    if delta < theta:
        break
    return V, it

# Sanity check: bajo la política aleatoria, un theta más estricto no debería
# cambiar V de forma perceptible, solo tomar más iteraciones.
_V_check, _it_check = policy_evaluation(
    mdp, {s: {a: 1.0 / len(mdp.actions) for a in mdp.actions} for s in mdp.
↪states},
    gamma=0.95, theta=1e-3)
print(f"Ejemplo: política aleatoria con theta=1e-3 converge en {_it_check}
↪iteraciones")

```

Ejemplo: política aleatoria con theta=1e-3 converge en 188 iteraciones

1.5.3 Inciso 3

Evaluar y comparar al menos tres políticas distintas: la política uniforme aleatoria, una política determinista fija definida por ustedes, y la política greedy derivada de los valores de la política anterior.

Las tres políticas:

1. **Aleatoria uniforme.** $\pi(a \mid s) = 1/5$ para las 5 acciones, en todo estado. Cota inferior de referencia.
2. **Fija determinista (regla de negocio).** Navegación por camino más corto (BFS sobre la cuadrícula, tratando los edificios como paredes) hacia la entrega no obstruida más cercana. Es una política “de manual de operación” — razonable y sin colisiones intencionales — pero ciega a la función de recompensa: no sabe que las celdas de viento y las adyacentes a edificios

son más riesgosas, solo minimiza distancia.

3. **Greedy derivada de la política anterior.** Un paso de mejora de política sobre $V^{\pi_{\text{fija}}}$: $\pi_{\text{greedy}}(s) = \arg \max_a \sum_{s'} p(s' | s, a) [r(s, a, s') + \gamma V^{\pi_{\text{fija}}}(s')]$. Por el teorema de mejora de política, $V^{\pi_{\text{greedy}}}(s) \geq V^{\pi_{\text{fija}}}(s)$ para todo s — pero al ser solo **un** paso de mejora (no policy iteration hasta convergencia), no está garantizado que alcance la política óptima global.

```
[4]: def random_policy(mdp):
    return {s: {a: 1.0 / len(mdp.actions) for a in mdp.actions} for s in mdp.
    ↪states}

def fixed_policy(mdp):
    """Política fija manual: camino más corto (BFS, edificios = paredes)
    hacia la entrega más cercana. Determinista, sin conocimiento de la
    recompensa ni del viento."""

    def bfs_next_step(start, goal):
        if start == goal:
            return WAIT
        visited = {start}
        queue = deque([start])
        parent = {}
        while queue:
            cur = queue.popleft()
            if cur == goal:
                break
            for a in (NORTH, SOUTH, EAST, WEST):
                nxt = mdp._attempt(cur, a)
                if nxt == cur or nxt in visited:
                    continue
                visited.add(nxt)
                parent[nxt] = (cur, a)
                queue.append(nxt)
            if goal not in parent:
                return WAIT
        node = goal
        while parent[node][0] != start:
            node = parent[node][0]
        return parent[node][1]

    policy = {}
    for s in mdp.states:
        if mdp.is_terminal(s):
            policy[s] = {a: 1.0 / len(mdp.actions) for a in mdp.actions}
            continue
        target = min(DELIVERIES, key=lambda d: abs(d[0] - s[0]) + abs(d[1] -
        ↪s[1]))
```

```

        chosen = bfs_next_step(s, target)
        policy[s] = {act: (1.0 if act == chosen else 0.0) for act in mdp.
↪actions}
        return policy

def greedy_action(mdp, V, s, gamma=0.95):
    if mdp.is_terminal(s):
        return None
    q = {a: sum(p * (r + gamma * V[s_next]) for p, s_next, r in mdp.
↪step_distribution(s, a))
        for a in mdp.actions}
    return max(q, key=q.get)

def greedy_policy_from_values(mdp, V, gamma=0.95):
    policy = {}
    for s in mdp.states:
        if mdp.is_terminal(s):
            policy[s] = {a: 1.0 / len(mdp.actions) for a in mdp.actions}
            continue
        best = greedy_action(mdp, V, s, gamma)
        policy[s] = {a: (1.0 if a == best else 0.0) for a in mdp.actions}
    return policy

GAMMA, THETA = 0.95, 1e-6 # gamma propuesto en la Tarea 1, Inciso 5

pi_random = random_policy(mdp)
V_random, it_random = policy_evaluation(mdp, pi_random, GAMMA, THETA)

pi_fixed = fixed_policy(mdp)
V_fixed, it_fixed = policy_evaluation(mdp, pi_fixed, GAMMA, THETA)

pi_greedy = greedy_policy_from_values(mdp, V_fixed, GAMMA)
V_greedy, it_greedy = policy_evaluation(mdp, pi_greedy, GAMMA, THETA)

# Verificación del teorema de mejora de política: greedy >= fija en todo estado.
min_improvement = min(V_greedy[s] - V_fixed[s] for s in mdp.states)
print(f"Mejora mínima de greedy sobre fija (debe ser >= 0): {min_improvement:.
↪6f}")

print(f"{'Política':<20}{'Iteraciones':>12}")
for name, it in [("Aleatoria", it_random), ("Fija (BFS)", it_fixed),
↪("Greedy(V_fija)", it_greedy)]:
    print(f"{name:<20}{it:>12}")

```

```

Mejora mínima de greedy sobre fija (debe ser >= 0): 0.000000
Política           Iteraciones
Aleatoria           302
Fija (BFS)          29
Greedy(V_fija)      271

```

1.5.4 Inciso 4

Producir para cada política evaluada: una tabla de valores $V^\pi(s)$ para todos los estados, el número de iteraciones hasta convergencia, y una visualización del grid con los valores codificados en color y las acciones de la política greedy superpuestas como flechas.

El número de iteraciones ya se listó en el Inciso 3. A continuación las tablas de $V^\pi(s)$ (una fila = una fila del grid) y las visualizaciones. En cada visualización, la flecha de cada celda es la acción **greedy de un paso respecto al $V^\pi(s)$ de esa misma política** ($\arg \max_a \sum_{s'} p(s' | s, a) [r + \gamma V^\pi(s')]$): es la forma estándar de superponer “la acción greedy” sobre un mapa de valores, y es la única bien definida para la política aleatoria (que no tiene una única acción por estado). Para la política fija y la greedy, estas flechas no son necesariamente su propia acción tabulada — muestran hacia dónde apunta la mejora local dado ese V^π , lo cual es en sí mismo informativo (p. ej. revela dónde la política fija ya es localmente óptima y dónde no).

```

[5]: def value_table(mdp, V, label):
    arr = np.array([[V[(r, c)] for c in range(mdp.size)] for r in range(mdp.
    ↪size)])
    df = pd.DataFrame(arr, index=[f"fila {r}" for r in range(mdp.size)],
                      columns=[f"col {c}" for c in range(mdp.size)]).round(2)
    print(f"V^pi(s) - {label}")
    display(df)
    return df

_ = value_table(mdp, V_random, "Aleatoria uniforme")
_ = value_table(mdp, V_fixed, "Fija (BFS)")
_ = value_table(mdp, V_greedy, "Greedy(V_fija)")

```

$V^\pi(s)$ - Aleatoria uniforme

	col 0	col 1	col 2	col 3	col 4
fila 0	-1223.37	-1542.06	-1734.97	-977.78	0.00
fila 1	-1221.36	-1369.82	-1852.88	-1171.38	-457.20
fila 2	-1009.18	-1218.18	-1400.49	-980.74	-556.83
fila 3	-848.31	-1033.86	-920.19	-711.41	-373.82
fila 4	-719.86	-775.59	-771.88	-439.72	0.00

$V^\pi(s)$ - Fija (BFS)

	col 0	col 1	col 2	col 3	col 4
fila 0	-33.50	-199.73	-181.00	21.99	0.00
fila 1	-25.05	-27.20	-160.13	-8.71	21.99
fila 2	3.89	-20.36	-109.85	-23.12	39.13

```

fila 3   6.63  -16.76   9.79 -21.95  46.04
fila 4   9.49   11.37  14.77  46.04   0.00

```

$V^{\pi}(s)$ - Greedy(V_{fija})

```

      col 0  col 1  col 2  col 3  col 4
fila 0 -14.71 -42.63 -20.00  21.99   0.00
fila 1  -9.15 -20.00 -20.00  -4.24  24.94
fila 2   7.83 -20.00 -20.00  13.47  43.49
fila 3  11.00   8.17  15.95  34.71  48.87
fila 4  12.98  15.01  17.44  48.87   0.00

```

```

[6]: ARROW_DELTA = {NORTH: (0, 0.3), SOUTH: (0, -0.3), EAST: (0.3, 0), WEST: (-0.3, 0)}

```

```

def plot_value_grid(mdp, V, title, gamma=0.95):
    size = mdp.size
    grid = np.array([V[(r, c)] for c in range(size)] for r in range(size))

    fig, ax = plt.subplots(figsize=(5.5, 5.5))
    vmax = max(np.max(np.abs(grid)), 1e-6)
    im = ax.imshow(grid, cmap="RdYlGn", vmin=-vmax, vmax=vmax)

    for r in range(size):
        for c in range(size):
            s = (r, c)
            if s in OBSTACLES:
                ax.add_patch(plt.Rectangle((c - 0.5, r - 0.5), 1, 1,
color="#333333"))
                continue

            ax.text(c, r + 0.28, f"{V[s]:.1f}", ha="center", va="center",
                    fontsize=8, color="black")

            if s in DELIVERIES:
                ax.text(c, r - 0.15, "D", ha="center", va="center",
                    fontsize=13, fontweight="bold", color="black")
                continue
            if s == BASE:
                ax.text(c - 0.35, r - 0.35, "B", ha="center", va="center",
                    fontsize=10, fontweight="bold", color="black")

    a = greedy_action(mdp, V, s, gamma)
    if a == WAIT:
        ax.plot(c, r - 0.12, marker="o", color="black", markersize=4)
    else:
        dx, dy = ARROW_DELTA[a]

```



```

        ax.annotate("", xy=(c + dx, r - 0.12 - dy), xytext=(c, r - 0.
↪12),
                                arrowprops=dict(arrowstyle="->", color="black", ↪
↪lw=1.6))

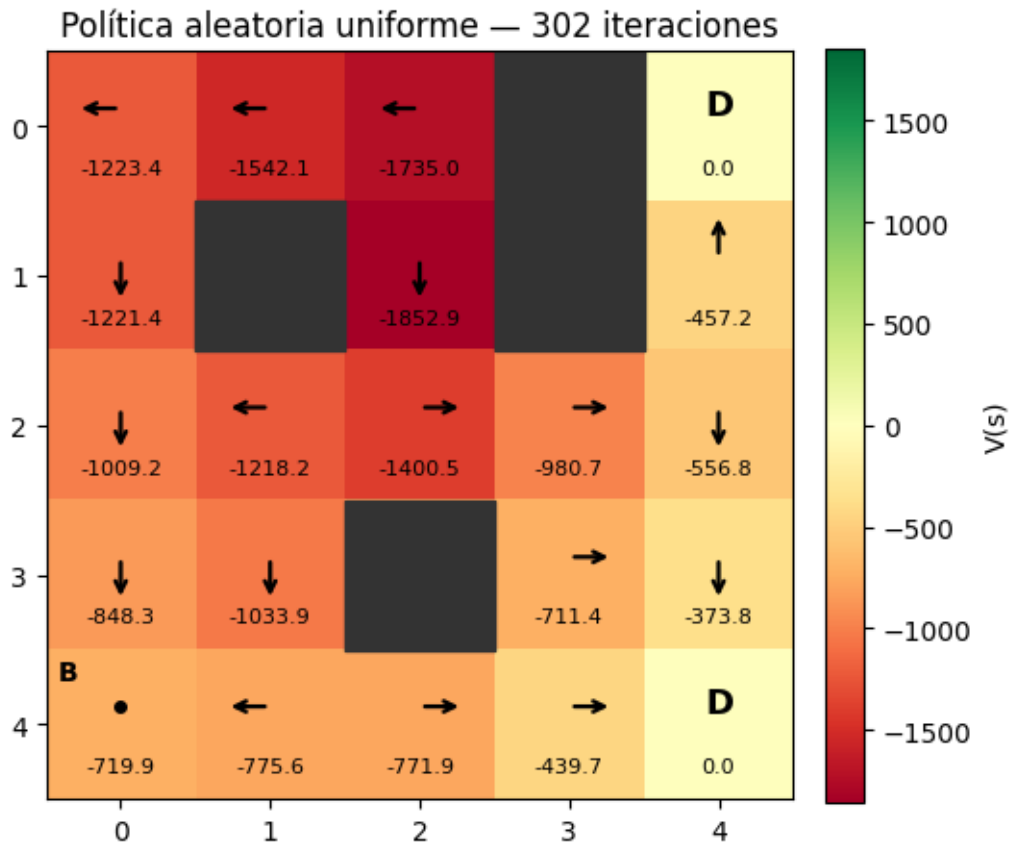
    ax.set_xticks(range(size))
    ax.set_yticks(range(size))
    ax.set_title(title)
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04, label="V(s)")
    fig.tight_layout()
    return fig

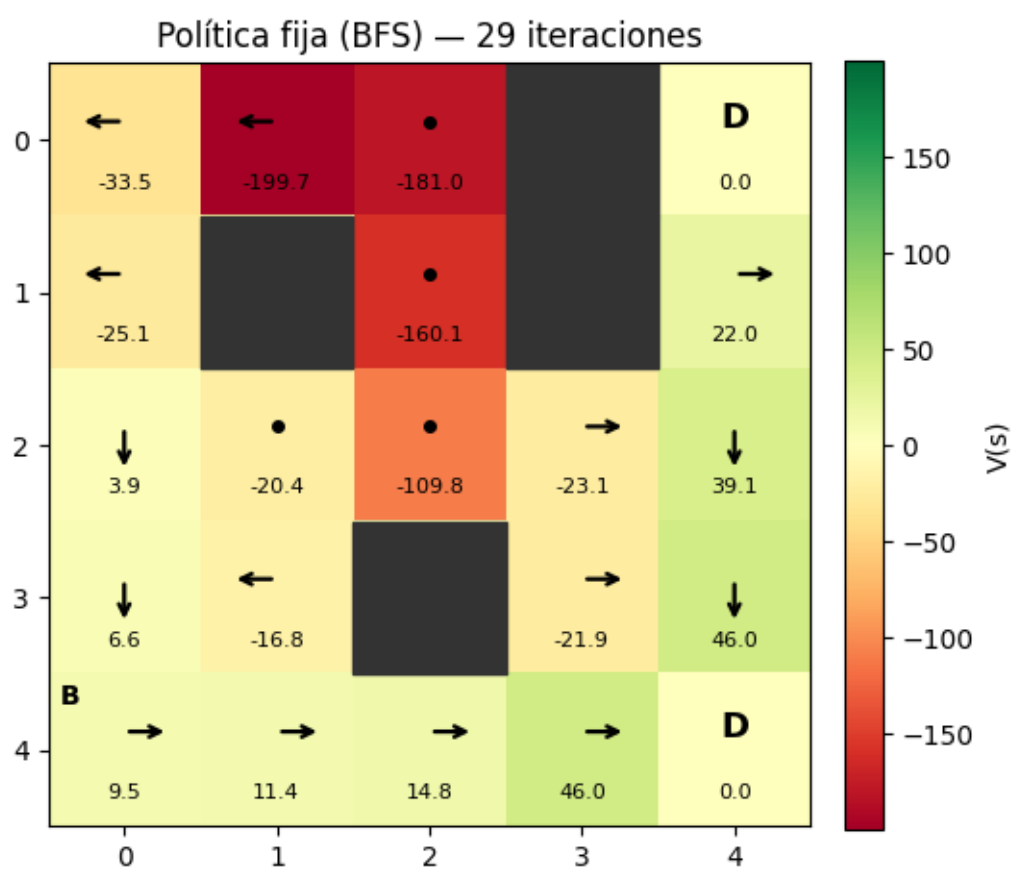
```

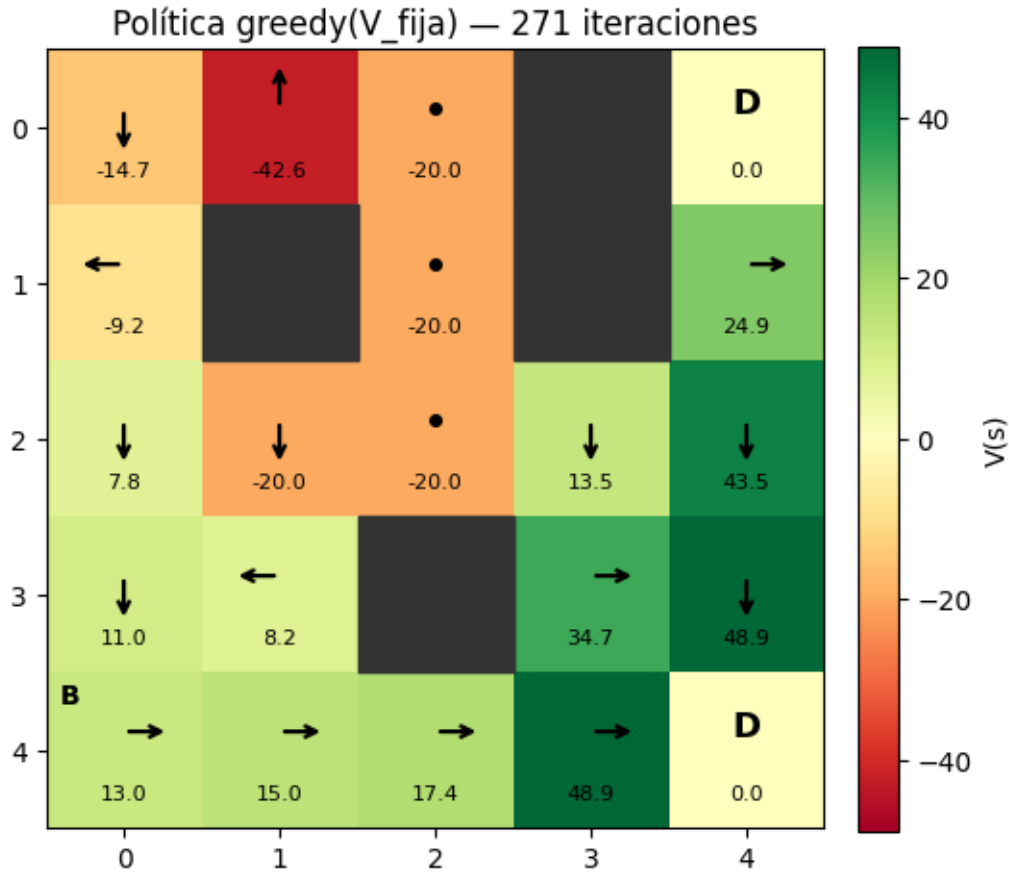
```

plot_value_grid(mdp, V_random, f"Política aleatoria uniforme - {it_random} ↪
↪iteraciones", GAMMA)
plot_value_grid(mdp, V_fixed, f"Política fija (BFS) - {it_fixed} iteraciones", ↪
↪GAMMA)
plot_value_grid(mdp, V_greedy, f"Política greedy(V_fija) - {it_greedy} ↪
↪iteraciones", GAMMA)
plt.show()

```







1.6 Task 4

Con base en los resultados de la implementación, redacten un dictamen técnico dirigido a la gerencia de la empresa de logística. El dictamen debe:

1.6.1 Inciso 1

Comparación cuantitativa de las políticas Para responder literalmente a este inciso se construye una política greedy a partir de los valores de la política aleatoria, además de las políticas aleatoria y fija ya evaluadas, esto es importante porque en la Tarea 3 también se mostró una greedy derivada de $V^{\pi_{fija}}$; ambas políticas greedy provienen de funciones de valor diferentes y no deben confundirse, las estadísticas se calculan sobre las 19 celdas transitables no terminales; se excluyen los cuatro edificios y las dos entregas absorbentes.

```
[7]: # Política greedy solicitada en la Task 4: derivada de V de la política
      ↪ aleatoria.
pi_greedy_random = greedy_policy_from_values(mdp, V_random, GAMMA)
V_greedy_random, it_greedy_random = policy_evaluation(
    mdp, pi_greedy_random, GAMMA, THETA
```

```

)

states_report = [s for s in mdp.states
                  if not mdp.is_terminal(s) and s not in OBSTACLES]

def policy_metrics(name, V, iterations):
    values = np.array([V[s] for s in states_report])
    return {
        "Política": name,
        "Iteraciones": iterations,
        "V(base)": V[BASE],
        "Promedio V": values.mean(),
        "Mínimo V": values.min(),
        "Máximo V": values.max(),
    }

comparison = pd.DataFrame([
    policy_metrics("Aleatoria uniforme", V_random, it_random),
    policy_metrics("Fija (BFS)", V_fixed, it_fixed),
    policy_metrics("Greedy de V_aleatoria", V_greedy_random, it_greedy_random),
]).set_index("Política").round(2)
display(comparison)

# Estados con mayor diferencia absoluta entre la greedy y la fija.
largest_differences = sorted(
    ((s, V_greedy_random[s], V_fixed[s],
      V_greedy_random[s] - V_fixed[s]) for s in states_report),
    key=lambda row: abs(row[3]), reverse=True
)[:8]
display(pd.DataFrame(
    largest_differences,
    columns=["Estado", "V greedy", "V fija", "Diferencia"]
).round(2))

```

		Iteraciones	V(base)	Promedio V	Mínimo V	Máximo V
Política						
Aleatoria uniforme		302	-719.86	-993.25	-1852.88	-373.82
Fija (BFS)		29	9.49	-31.16	-199.73	46.04
Greedy de V_aleatoria		271	-20.00	-19.36	-128.33	48.87

	Estado	V greedy	V fija	Diferencia
0	(0, 1)	-61.53	-199.73	138.20
1	(0, 2)	-62.68	-181.00	118.32
2	(3, 3)	34.71	-21.95	56.66
3	(2, 3)	13.47	-23.12	36.60
4	(2, 2)	-76.18	-109.85	33.67
5	(4, 1)	-21.37	11.37	-32.75
6	(1, 2)	-128.33	-160.13	31.81

7 (3, 1) -47.55 -16.76 -30.79

Con $\gamma = 0.95$, la greedy derivada de la aleatoria obtiene el promedio más alto, -19.36 , seguida por la fija con -31.16 y, muy lejos, la aleatoria con -993.25 , desde la base $(4, 0)$, sin embargo, la fija es claramente superior: 9.49 frente a -20.00 de la greedy y -719.86 de la aleatoria. La política aleatoria también requiere 302 iteraciones, contra 29 de la fija y 271 de la greedy.

Las mayores mejoras de la greedy respecto de la fija aparecen en $(0, 1)$ ($+138.20$), $(0, 2)$ ($+118.32$) y $(3, 3)$ ($+56.66$), las mayores pérdidas aparecen en $(4, 1)$ (-32.75), $(3, 1)$ (-30.79) y $(2, 1)$ (-29.71), frente a la aleatoria, la mejora más grande ocurre en $(1, 2)$: el valor pasa de -1852.88 a -128.33 .

La greedy derivada de la aleatoria no es inequívocamente mejor que la política fija, tiene mayor valor en 10 de las 19 celdas evaluadas, la fija es mayor en 8 y empatan en una, su mejor promedio convive con un peor desempeño desde la base, el motivo es que una sola mejora greedy sobre $V^{\pi_{\text{aleatoria}}}$ hereda valores extremadamente negativos cerca de edificios y puede seleccionar Esperar indefinidamente, en la base, esta decisión produce exactamente $-1/(1 - 0.95) = -20$.

Por tanto, para operación interesa más la comparación desde la distribución real de estados iniciales —principalmente la base— que el promedio uniforme de toda la cuadrícula. Bajo ese criterio, la política fija es preferible a esta greedy de un solo paso.

1.6.2 Inciso 2

Sensibilidad al factor de descuento Se repite el experimento con $\gamma = 0.80$ y $\gamma = 0.99$, además del valor propuesto $\gamma = 0.95$. Para cada valor se vuelven a evaluar la política aleatoria, la fija y la greedy obtenida mediante una mejora de un paso sobre $V^{\pi_{\text{aleatoria}}}$.

```
[8]: sensitivity_rows = []
greedy_actions_by_gamma = {}

for gamma_test in (0.80, 0.95, 0.99):
    V_r, it_r = policy_evaluation(mdp, pi_random, gamma_test, THETA)
    V_f, it_f = policy_evaluation(mdp, pi_fixed, gamma_test, THETA)
    pi_g = greedy_policy_from_values(mdp, V_r, gamma_test)
    V_g, it_g = policy_evaluation(mdp, pi_g, gamma_test, THETA)

    greedy_actions_by_gamma[gamma_test] = {
        s: greedy_action(mdp, V_r, s, gamma_test) for s in states_report
    }
    for name, V, iterations in (
        ("Aleatoria", V_r, it_r),
        ("Fija (BFS)", V_f, it_f),
        ("Greedy de V_aleatoria", V_g, it_g),
    ):
        sensitivity_rows.append({
            "gamma": gamma_test,
            "Política": name,
            "Iteraciones": iterations,
            "V(base)": V[BASE],
            "Promedio V": np.mean([V[s] for s in states_report]),
```

```

    })

sensitivity = pd.DataFrame(sensitivity_rows)
display(sensitivity.set_index(["gamma", "Política"]).round(2))

reference_actions = greedy_actions_by_gamma[0.95]
action_changes = []
for gamma_test in (0.80, 0.99):
    changed = [s for s in states_report
                if greedy_actions_by_gamma[gamma_test][s] !=
↪reference_actions[s]]
    action_changes.append({
        "gamma comparado": gamma_test,
        "Cambios respecto a gamma=0.95": len(changed),
        "Estados que cambian": changed,
    })
display(pd.DataFrame(action_changes))

```

		Iteraciones	V(base)	Promedio V
gamma	Política			
0.80	Aleatoria	80	-97.07	-292.47
	Fija (BFS)	24	1.55	-26.05
	Greedy de V_aleatoria	63	-5.00	-1.81
0.95	Aleatoria	302	-719.86	-993.25
	Fija (BFS)	29	9.49	-31.16
	Greedy de V_aleatoria	271	-20.00	-19.36
0.99	Aleatoria	931	-2914.24	-2940.87
	Fija (BFS)	31	12.60	-32.45
	Greedy de V_aleatoria	29	15.28	-16.05

	gamma comparado	Cambios respecto a gamma=0.95	\
0	0.80		3
1	0.99		6

	Estados que cambian
0	[(0, 0), (2, 2), (3, 1)]
1	[(0, 0), (0, 2), (2, 1), (2, 4), (4, 0), (4, 1)]

El efecto de γ depende de la estructura de cada política:

- **Aleatoria:** al aumentar γ , sus costos futuros recurrentes pesan más. El valor de la base cae de -97.07 ($\gamma = 0.80$) a -719.86 (0.95) y -2914.24 (0.99); el promedio cae de -292.47 a -993.25 y -2940.87 . La convergencia también se ralentiza: 80, 302 y 931 iteraciones.
- **Fija:** el valor de la base sube de 1.55 a 9.49 y 12.60, porque un γ alto preserva más del premio futuro de entrega. Su promedio, no obstante, pasa de -26.05 a -31.16 y -32.45 : en estados riesgosos también se propagan durante más tiempo las penalizaciones. Converge rápidamente en 24, 29 y 31 iteraciones.
- **Greedy de la aleatoria:** con $\gamma = 0.80$ y 0.95 escoge Esperar en la base, dando -5.00 y -20.00 , respectivamente. Con $\gamma = 0.99$ la recompensa distante de entrega pesa lo suficiente

para que escoja Este y el valor de la base suba a 15.28. Su promedio es -1.81 , -19.36 y -16.05 .

La política greedy sí es sensible al descuento: respecto de $\gamma = 0.95$, cambian 3 acciones con $\gamma = 0.80$ y 6 con $\gamma = 0.99$, el hallazgo operativo es que γ no debe elegirse solo por convergencia matemática; debe calibrarse con la duración real de las rutas y comprobarse que no incentive la inacción, el valor 0.95 representa bien un horizonte aproximado de 20 pasos, pero la greedy de un solo paso todavía requiere más iteraciones de mejora o una restricción contra Esperar indefinidamente.

1.6.3 Inciso 3

Limitaciones frente a la logística urbana real El GridWorld implementado es útil para validar el operador de Bellman, pero no representa todavía un sistema de reparto operacional:

1. **Estado reducido.** El código usa únicamente (x, y) . La batería, entregas pendientes, carga y clima propuestas en la Tarea 1 se convirtieron en propiedades fijas del terreno. Por ello no puede distinguir dos visitas a una celda con distinta batería, paquete o progreso. La extensión mínima es restaurar $s = (x, y, b, \mathbf{d}, c, w)$.
2. **Observabilidad perfecta.** Se supone que posición, mapa y viento se conocen sin error. En la ciudad hay ruido de GPS, viento local no observado y obstáculos temporales. Se necesita un POMDP o, como aproximación, un estado de creencia estimado con fusión de sensores.
3. **Mapa estático y discreto.** Los edificios, entregas y celdas de viento no cambian; tampoco existen altitud, velocidad, orientación ni aceleración. Un modelo útil requiere un mapa tridimensional, dinámica temporal de zonas restringidas y acciones compatibles con la cinemática del dron.
4. **Un solo dron y dos terminales absorbentes.** No se modelan otros drones, peatones, tráfico aéreo, capacidad de carga, múltiples paquetes, recogidas ni retorno obligatorio a la base. Hacen falta estados conjuntos o coordinación multiagente, además de una misión terminal que exija completar entregas y regresar.
5. **Probabilidades y recompensas hipotéticas.** Los valores de viento, fallo y colisión no provienen de telemetría. Antes del despliegue deben estimarse $p(s' | s, a)$ y costos con datos históricos, pruebas controladas y criterios económicos de la empresa.
6. **Seguridad tratada como penalización.** Una recompensa de -500 reduce colisiones en promedio, pero no ofrece garantías. Las restricciones críticas —batería de reserva, geocercas, separación y límites meteorológicos— deben imponerse como acciones inválidas o mediante un MDP restringido (*Constrained MDP*), no dejarse únicamente al balance de recompensas.
7. **Ausencia de validación fuera del modelo.** Un alto V^π solo es válido bajo las transiciones supuestas. Se requieren simulación de alta fidelidad, análisis de robustez ante parámetros inciertos y pruebas de escenarios extremos.

1.6.4 Inciso 4

Dictamen y recomendación a gerencia El MDP es una base conceptual y de prototipado suficiente, pero no es una base operacional suficiente para desplegar un sistema de RL real, la implementación demuestra correctamente que una política informada supera ampliamente a la exploración aleatoria y permite estudiar de manera transparente transiciones, recompensas y descuento, sin embargo, la greedy derivada de la aleatoria puede preferir esperar indefinidamente y no domina a la política manual desde la base. Este resultado confirma que aprobar el sistema por el promedio global de $V(s)$ sería riesgoso.

Se recomienda avanzar por etapas:

1. Restaurar el estado completo de la Tarea 1 y definir con precisión las condiciones terminales: entregas completadas, retorno seguro, batería agotada o aborto de misión.
2. Recopilar telemetría y datos de operación para calibrar probabilidades, consumo, fallos y costos; validar la propiedad de Markov y decidir si se requiere un POMDP.
3. Sustituir la mejora greedy de un solo paso por iteración de políticas hasta estabilidad o iteración de valores, y comparar contra BFS y otros planificadores usando la distribución real de salidas desde la base.
4. Separar objetivos económicos de restricciones duras de seguridad mediante un MDP restringido y un supervisor de seguridad independiente.
5. Validar primero en simulación de alta fidelidad con clima, sensores, tráfico y múltiples rutas; luego realizar pruebas *hardware-in-the-loop*, vuelos controlados sin carga y un piloto supervisado.
6. Definir criterios de aprobación antes del despliegue: tasa de entrega, energía por misión, retorno con reserva, probabilidad de violación de seguridad, robustez ante viento y desempeño frente a la política manual.

La recomendación final es continuar la investigación, pero no desplegar todavía, el siguiente hito debe ser un simulador calibrado con datos reales en el que la política optimizada supere consistentemente a la política manual desde estados operacionales relevantes y cumpla restricciones de seguridad verificables.

1.7 Referencias

- [An Introduction to Markov Decision Processes](#)

Tratamiento formal de la tupla, la propiedad de Markov y las condiciones de existencia de política óptima.

- [Understanding the Markov Decision Process \(MDP\)](#)

Enfoque aplicado al modelado de problemas reales como MDP.

- [Markov Decision Processes \(MDPs\) - Structuring a Reinforcement Learning Problem](#)

Criterios prácticos para estructurar estado, acción y recompensa en un problema concreto.